GLOBALRAIN

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY 3
CLIENT 3
INSTRUCTIONS 3
DEVELOPER 4
1. ALGORITHM CIPHER 4
2. CERTIFICATE GENERATION 5
3. DEPLOY CIPHER 5
4. SECURE COMMUNICATIONS 5
5. SECONDARY TESTING 6
6. FUNCTIONAL TESTING 7
7. SUMMARY 7
8. INDUSTRY STANDARD BEST PRACTICES 8

Document Revision History

Version	Date	Author	Comments
1.0	2/19	Jesus Rodriguez-Luna	

Client

Instructions

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

Developer
Jesus Rodriguez-Luna

1. Algorithm Cipher

Artemis Financial's web application needs a secure way to make sure files are not changed while being transferred. To solve this, the best option is SHA 256. SHA 256 is a secure hashing algorithm that creates a unique fingerprint for any file or piece of data. If even one character changes, the fingerprint changes completely. That makes it perfect for checking file integrity.

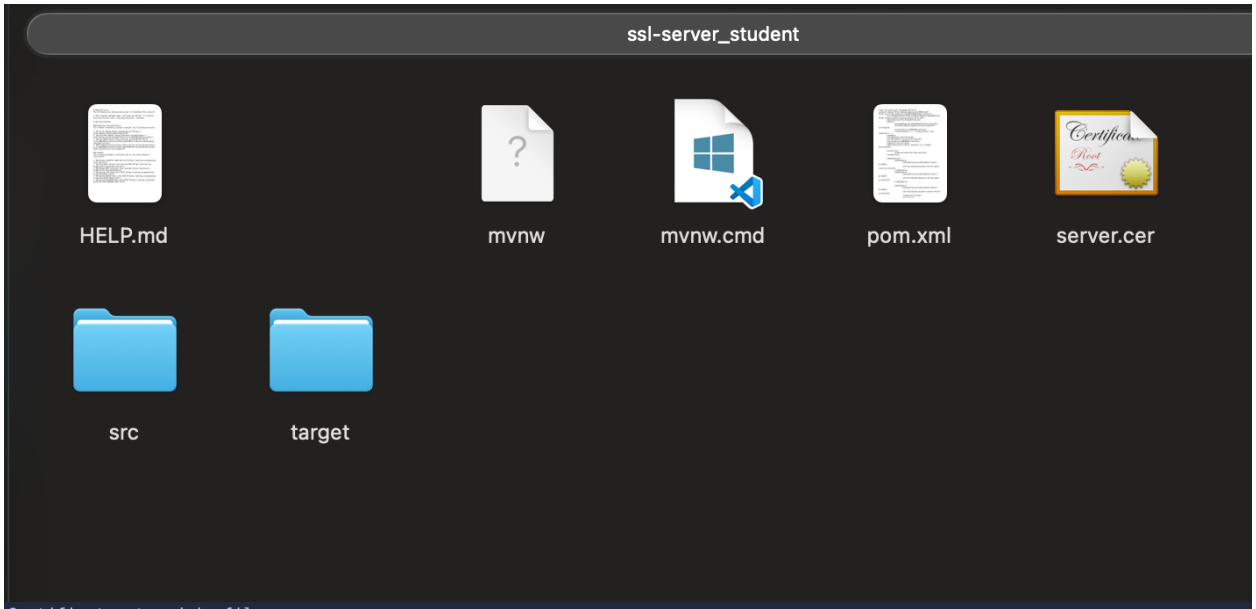
SHA 256 takes any amount of data and turns it into a fixed 256 bit value. This value is usually shown as a 64 character string made of letters and numbers. The important thing to understand is that SHA 256 is one way. You can create a hash from data, but you cannot reverse it to get the original data back. The same input will always produce the same output, which makes it reliable for verification.

It works by processing data in blocks and running multiple rounds of mathematical operations. The 256 bit length makes it extremely difficult for attackers to create two different inputs that produce the same hash. Older algorithms like MD5 and SHA 1 were once common, but they are no longer considered secure because researchers were able to find collision vulnerabilities. SHA 256 does not have those weaknesses and is still trusted today.

It is also important to note that SHA 256 is not encryption. It does not use keys and it does not hide data. Encryption algorithms like AES or RSA are used when you need to protect data so only authorized users can read it. SHA 256 is used to verify integrity. If Artemis Financial wanted to verify both integrity and authenticity, they could use HMAC SHA 256, which adds a secret key into the process. For this project, basic SHA 256 is enough to generate and compare checksums.

Over time, hashing algorithms have improved as older ones became vulnerable. SHA 256 has been heavily tested for years and remains secure. It is widely used in modern security systems, including TLS certificates and even Bitcoin. Since Java already supports SHA 256 through the MessageDigest class, it can be implemented easily in a Spring Boot application without needing extra libraries.

2. Certificate Generation




```
Certificate stored in file <server.cer>
❯ keytool -printcert -file server.cer


Owner: CN=Jesus Rodriguez-Luna, OU=SNHU, O=SNHU, L=Pomona, ST=CA, C=US
Issuer: CN=Jesus Rodriguez-Luna, OU=SNHU, O=SNHU, L=Pomona, ST=CA, C=US
Serial number: 96d1b60a55deafa6
Valid from: Thu Feb 19 03:35:26 PST 2026 until: Sun Feb 14 03:35:26 PST 2027
Certificate fingerprints:
         SHA1: 67:81:06:B4:54:CB:A6:6E:7C:83:78:18:74:E4:12:D2:6A:EC:97:2C
         SHA256: A1:91:DD:96:69:03:03:3E:ED:59:D5:15:95:DE:38:6C:28:2B:A2:53:80:18:C7:60:02:CE:6A:78:19:25:58:C6
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: 0B 81 A1 3C D5 D3 94 E7   7D 78 0F B5 9A 8F 36 32  ...<.....x....62
0010: 82 1C 5A 1D                                        ..Z.
]
]
```

3. Deploy Cipher

data:Jesus Rodriguez-Luna!

SHA-256: Checksum Value: 0af7c7ec8c1716fc0f181ee3eb08f014da19a88c044654bad790a43b9bb0c25e

4. Secure Communications

5. Secondary Testing

```
module_seven [Maven Build] /Users/jrodriguez-luna/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.aarch64_21.0.9.v20251105-0741/jre/bin/java  (Feb 19, 2026, 4:30:35 AM elapsed 0:04:07) [pid: 39969]
[INFO] Scanning for projects...
[INFO]
[INFO] -----------------------< com.snhu:ssl-server >------------------------
[INFO] Building ssl-server 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] >>> spring-boot:2.2.4.RELEASE:run (default-cli) > test-compile @ ssl-server >>>
[INFO]
[INFO] --- resources:3.1.0:resources (default-resources) @ ssl-server ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 1 resource
[INFO] Copying 1 resource
[INFO]
[INFO] --- compiler:3.8.1:compile (default-compile) @ ssl-server ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.1.0:testResources (default-testResources) @ ssl-server ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /Applications/School/CS305 - Software Security/Module Seven/ssl-server_student/src/test/resources
[INFO]
[INFO] --- compiler:3.8.1:testCompile (default-testCompile) @ ssl-server ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] <<< spring-boot:2.2.4.RELEASE:run (default-cli) < test-compile @ ssl-server <<<
[INFO]
[INFO]
[INFO] --- spring-boot:2.2.4.RELEASE:run (default-cli) @ ssl-server ---
[INFO] Attaching agents: []

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.2.4.RELEASE)

2026-02-19 04:30:36.678  INFO 39970 --- [           main] com.snhu.sslserver.SslServerApplication  : Starting SslServerApplication on DevBook-Pro.local with PID 39970 (started by j
2026-02-19 04:30:36.679  INFO 39970 --- [           main] com.snhu.sslserver.SslServerApplication  : No active profile set, falling back to default profiles: default
2026-02-19 04:30:36.954  INFO 39970 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8443 (https)
2026-02-19 04:30:36.957  INFO 39970 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-02-19 04:30:36.957  INFO 39970 --- [           main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/9.0.30]
2026-02-19 04:30:36.980  INFO 39970 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2026-02-19 04:30:36.980  INFO 39970 --- [           main] o.s.web.context.ContextLoader            : Root WebApplicationContext: initialization completed in 289 ms
2026-02-19 04:30:37.107  INFO 39970 --- [           main] o.s.s.concurrent.ThreadPoolTaskExecutor  : Initializing ExecutorService 'applicationTaskExecutor'
2026-02-19 04:30:37.279  INFO 39970 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8443 (https) with context path ''
2026-02-19 04:30:37.280  INFO 39970 --- [           main] com.snhu.sslserver.SslServerApplication  : Started SslServerApplication in 0.685 seconds (JVM running for 0.777)
2026-02-19 04:30:39.408  INFO 39970 --- [nio-8443-exec-3] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-02-19 04:30:39.408  INFO 39970 --- [nio-8443-exec-3] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
2026-02-19 04:30:39.413  INFO 39970 --- [nio-8443-exec-3] o.s.web.servlet.DispatcherServlet        : Completed initialization in 5 ms
```

DEPENDENCY-CHECK

Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for ar damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

How to read the report | Suppressing false positives | Getting Help: github issues

Project: ssl-server

com.snhu:ssl-server:0.0.1-SNAPSHOT

Scan Information (show all):

- *dependency-check version*: 12.2.0
- *Report Generated On*: Thu, 19 Feb 2026 04:09:21 -0800
- *Dependencies Scanned*: 49 (28 unique)
- *Vulnerable Dependencies*: 13
- *Vulnerabilities Found*: 148
- *Vulnerabilities Suppressed*: 0
- ...

Summary

Summary of Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:*:*:*:*:* cpe:2.3:a:redhat:hibernate_validator:6.0.18:*:*:*:*:*:*:*	pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final	MEDIUM	3	Highest	32
jackson-databind-2.10.2.jar	cpe:2.3:a:fasterxml:jackson-databind:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-modules-java8:2.10.2:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.10.2	HIGH	6	Highest	39
json-path-2.4.0.jar	cpe:2.3:a:json-path:jayway_jsonpath:2.4.0:*:*:*:*:*:*:*	pkg:maven/com.jayway.jsonpath/json-path@2.4.0	MEDIUM	1	Highest	33
json-smart-2.3.jar	cpe:2.3:a:json-smart_project:json-smart:2.3:*:*:*:*:*:*:* cpe:2.3:a:json-smart_project:json-smart-v2:2.3:*:*:*:*:*:*:*	pkg:maven/net.minidev/json-smart@2.3	HIGH	2	Highest	45
log4j-api-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1	MEDIUM	2	Highest	42
logback-core-1.2.3.jar	cpe:2.3:a:qos:logback:1.2.3:*:*:*:*:*:*:*	pkg:maven/ch.qos.logback/logback-core@1.2.3	HIGH	2	Highest	31
snakeyaml-1.25.jar	cpe:2.3:a:snakeyaml_project:snakeyaml:1.25:*:*:*:*:*:*:*	pkg:maven/org.yaml/snakeyaml@1.25	CRITICAL	8	Highest	44
spring-boot-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*	pkg:maven/org.springframework.boot/spring-boot@2.2.4.RELEASE	CRITICAL	3	Highest	39
spring-core-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*:*	pkg:maven/org.springframework/spring-core@5.2.3.RELEASE	CRITICAL*	12	Highest	36
spring-hateoas-1.0.3.RELEASE.jar	cpe:2.3:a:vmware:spring_hateoas:1.0.3:release:*:*:*:*:*	pkg:maven/org.springframework.hateoas/spring-hateoas@1.0.3.RELEASE	MEDIUM	1	Highest	43
spring-web-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*:*	pkg:maven/org.springframework/spring-web@5.2.3.RELEASE	CRITICAL*	13	Highest	34
tomcat-embed-core-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.30	CRITICAL*	47	Highest	30
tomcat-embed-websocket-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-websocket@9.0.30	CRITICAL*	48	Highest	30

* indicates the dependency has a known exploited vulnerability

Dependencies (vulnerable)

hibernate-validator-6.0.18.Final.jar

Description:

Hibernate's Bean Validation (JSR-380) reference implementation.

6. Functional Testing

```java
package com.snhu.sslserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

@SpringBootApplication
public class SslServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(SslServerApplication.class, args);
    }

}

@RestController
class ServerController{
    public static String bytesToHex(byte[] bytes) {
        StringBuilder sb = new StringBuilder();
        for (byte b : bytes) {
            sb.append(String.format("%02x", b));
        }
        return sb.toString();
    }


    @RequestMapping("/hash")
    public String myHash() throws NoSuchAlgorithmException {
        String data = "Jesus Rodriguez-Luna!";

        MessageDigest digest = MessageDigest.getInstance("SHA-256");

        byte[] hashBytes = digest.digest(data.getBytes());

        String hexHash = bytesToHex(hashBytes);

        return "<p>data:" + data + "</p>" +"<p>SHA-256: Checksum Value: " + hexHash + "</p>";
    }
}
```

```
Module_seven [Maven Build] /Users/jrodriguez-luna/.p2/pool/plugins/org.eclipse.justj.openjdk.hotspot.jre.full.macosx.aarch64_21.0.9.v20251105-0741/jre/bin/java  (Feb 19, 2026, 4:30:35 AM elapsed 0:09:30) [pid: 39969]
[INFO]
[INFO] --- compiler:3.8.1:compile (default-compile) @ ssl-server ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.1.0:testResources (default-testResources) @ ssl-server ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] skip non existing resourceDirectory /Applications/School/CS305 - Software Security/Module Seven/ssl-server_student/src/test/resources
[INFO]
[INFO] --- compiler:3.8.1:testCompile (default-testCompile) @ ssl-server ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] <<< spring-boot:2.2.4.RELEASE:run (default-cli) < test-compile @ ssl-server <<<
[INFO]
[INFO]
[INFO] --- spring-boot:2.2.4.RELEASE:run (default-cli) @ ssl-server ---
[INFO] Attaching agents: []

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.2.4.RELEASE)

2026-02-19 04:30:36.678  INFO 39970 --- [           main] com.snhu.sslserver.SslServerApplication  : Starting SslServerApplication on DevBook-Pro.local with PID 39970 (started by jrodriguez-luna in /Applications/School/CS305 - Software Security/Mod
2026-02-19 04:30:36.679  INFO 39970 --- [           main] com.snhu.sslserver.SslServerApplication  : No active profile set, falling back to default profiles: default
2026-02-19 04:30:36.954  INFO 39970 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8443 (https)
2026-02-19 04:30:36.957  INFO 39970 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-02-19 04:30:36.957  INFO 39970 --- [           main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/9.0.30]
2026-02-19 04:30:36.980  INFO 39970 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2026-02-19 04:30:36.980  INFO 39970 --- [           main] o.s.web.context.ContextLoader            : Root WebApplicationContext: initialization completed in 289 ms
2026-02-19 04:30:37.107  INFO 39970 --- [           main] o.s.s.concurrent.ThreadPoolTaskExecutor  : Initializing ExecutorService 'applicationTaskExecutor'
2026-02-19 04:30:37.279  INFO 39970 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8443 (https) with context path ''
2026-02-19 04:30:37.280  INFO 39970 --- [           main] com.snhu.sslserver.SslServerApplication  : Started SslServerApplication in 0.685 seconds (JVM running for 0.777)
2026-02-19 04:30:39.408  INFO 39970 --- [nio-8443-exec-3] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-02-19 04:30:39.408  INFO 39970 --- [nio-8443-exec-3] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
2026-02-19 04:30:39.413  INFO 39970 --- [nio-8443-exec-3] o.s.web.servlet.DispatcherServlet        : Completed initialization in 5 ms
```

7. Summary

I refactored the code to make it more secure and ensure it aligns with security testing protocols. Referring to the vulnerability assessment process flow diagram, I focused on two main areas: data integrity and secure communications. For data integrity, I added SHA-256 hashing so that any data sent through the app can be verified and checked for tampering. For secure communications, I switched the app from HTTP to HTTPS, encrypting all traffic between the client and server using a self-signed SSL certificate generated with Java Keytool.

The process of adding these security layers was straightforward: I updated the application.properties file to enable HTTPS and set the keystore details, generated and added a self-signed certificate, implemented SHA-256 hashing in the ServerController class with a /hash endpoint, and finally ran the OWASP Dependency-Check Maven plugin to make sure the changes didn't introduce any new vulnerabilities. This confirmed that all existing vulnerabilities came from third-party dependencies and not the refactored code.

8. Industry Standard Best Practices

Several industry standard best practices were applied when refactoring Artemis Financial's application to improve its security. First, SHA 256 hashing was added, which follows NIST's recommendations and the FIPS 180 4 standard. Using Java's built in MessageDigest API instead of a third party library also avoids adding unnecessary dependencies that could bring their own vulnerabilities.

Second, the app was switched from HTTP to HTTPS, which is standard practice for protecting data in transit. HTTPS with TLS prevents eavesdropping and man in the middle attacks and is essential for any application handling sensitive financial information. Third, a self signed certificate was created using Java Keytool. While this works for development and testing, in production it would be replaced by a certificate from a trusted Certificate Authority.

As part of the secure development lifecycle, the OWASP Dependency Check tool was run to catch any known vulnerabilities in third party dependencies. This ensured that the refactoring did not introduce new security issues and aligns with OWASP's guidance on monitoring for outdated or vulnerable components.

These best practices add real value for Artemis Financial. Handling sensitive client data means that breaches could result in legal problems, fines, reputational damage, and loss of trust. By using hashing for data integrity, HTTPS for secure communications, and dependency checks for ongoing monitoring, the app meets baseline security expectations for financial software. These measures also support Global Rain's mission that security is everyone's responsibility, building security into development instead of treating it as an afterthought.